Camadas e responsabilidades em aplicações back-end

Objetivos da aula

- Nesta aula vamos compreender como aplicações back-end modernas são organizadas em camadas.
- Você entenderá o papel conceitual de Controller, Service, Repository e Model.
- Ao final, será capaz de explicar como essa organização garante separação de responsabilidades, modularidade e testabilidade.

Por que organizar em camadas?

- Sistemas modernos possuem múltiplas responsabilidades simultâneas.
- Misturar regras de negócio, acesso a dados e controle de requisições gera acoplamento excessivo.
- A organização em camadas surge para estruturar essas responsabilidades.

Separação de Responsabilidades

- Separação de responsabilidades significa dividir o sistema em partes com funções bem definidas.
- Cada camada executa apenas o que lhe compete.
- Isso reduz dependências desnecessárias entre componentes.

Estrutura Clássica em Camadas

- Uma aplicação back-end tradicional costuma ser dividida em:
  - Controller Layer
  - Service Layer
  - Repository Layer
  - Model Layer

Cada uma desempenha um papel específico na arquitetura.

Visão Geral da Comunicação

- O fluxo segue uma direção lógica.
- A requisição entra pelo Controller.
- Passa pelo Service.
- Chega ao Repository.
- E interage com o banco por meio do Model.

Essa cadeia mantém organização e previsibilidade.

Controller Layer

- A Controller Layer é responsável por receber requisições externas.
- Ela interpreta métodos HTTP e parâmetros enviados pelo cliente.
- Seu papel é orquestrar a chamada para a camada de serviço.

Responsabilidades do Controller

- Mapear endpoints
- Receber dados da requisição
- Retornar respostas apropriadas

O Controller não deve conter regras de negócio complexas.

O Controller como Ponte

- O Controller atua como intermediário entre cliente e sistema interno.
- Ele converte dados externos em estruturas compreensíveis pelo sistema.
- Também transforma o retorno interno em resposta HTTP.

Service Layer

- A Service Layer concentra as regras de negócio da aplicação.
- É nela que decisões e validações são aplicadas.
- Ela coordena operações entre múltiplos repositórios quando necessário.

Papel Estratégico do Service

- O Service representa o coração da aplicação.
- Ele implementa a lógica que diferencia o sistema de outros.
- Sem essa camada, a aplicação se torna apenas um intermediário de dados.

O Service e a Regra de Negócio

- Regras de negócio envolvem validações, cálculos e políticas internas.
- Essas regras não devem estar no Controller nem no Repository.
- Centralizá-las no Service melhora organização e manutenção.

Repository Layer

- A Repository Layer é responsável pela persistência de dados.
- Ela abstrai a comunicação com o banco de dados.
- Seu foco é armazenar, buscar, atualizar e remover informações.

Responsabilidades do Repository

- Executar consultas
- Persistir entidades
- Remover registros

Não deve conter lógica de negócio.

Abstração de Persistência

- O Repository isola detalhes do banco de dados.
- A camada Service não precisa saber como a consulta é feita.
- Isso permite trocar tecnologias de banco com menor impacto.

Model Layer

- A Model Layer representa os dados da aplicação.
- Ela define entidades que refletem conceitos do domínio.
- Essas entidades geralmente correspondem a tabelas no banco.

Entidades como Representação do Domínio

- Uma entidade representa um conceito real do sistema.
- Por exemplo, Cliente, Pedido ou Produto.
- Essas classes armazenam atributos e podem conter validações simples.

Comunicação Entre Camadas

- Controller chama Service.
- Service chama Repository.
- Repository manipula Model e banco.

Essa hierarquia evita dependências invertidas.

Direção do Fluxo

- A comunicação deve ser unidirecional.
- Camadas inferiores não devem depender de camadas superiores.
- Essa regra mantém estabilidade arquitetural.

Benefícios da Arquitetura em Camadas

- Organização estrutural
- Facilidade de manutenção
- Maior clareza de responsabilidades
- Melhor testabilidade

Esses benefícios tornam o sistema mais sustentável.

Modularidade

- Cada camada pode evoluir independentemente.
- Mudanças no banco não devem afetar o Controller diretamente.
- Essa independência reduz impacto de alterações.

Testabilidade

- Com responsabilidades separadas, é possível testar camadas isoladamente.
- O Service pode ser testado sem depender do banco real.
- Isso melhora qualidade e confiabilidade do software.

Escalabilidade Conceitual

- Aplicações bem organizadas crescem com menos caos estrutural.
- Novas funcionalidades podem ser adicionadas respeitando o padrão.
- A arquitetura permanece consistente ao longo do tempo.

Problemas de Misturar Camadas

- Quando regras de negócio ficam no Controller, o código se torna frágil.
- Quando o Repository contém validações, há violação de responsabilidade.
- Essa mistura gera manutenção difícil.

Acoplamento Excessivo

- Acoplamento ocorre quando componentes dependem demais uns dos outros.
- Arquitetura em camadas reduz esse problema.
- Ela estabelece fronteiras claras entre responsabilidades.

Coesão

- Coesão significa que cada módulo faz bem apenas uma coisa.
- A camada Service deve ser altamente coesa em regras de negócio.
- A camada Repository deve ser coesa em persistência.

Evolução Arquitetural

- A arquitetura em camadas é base para modelos mais complexos.
- Ela influencia arquiteturas como Clean Architecture.
- Também fundamenta princípios de design orientado a domínio.

Integração com Frameworks

- Frameworks modernos incentivam essa organização.
- O Spring Boot estrutura projetos com essa divisão.
- Isso reforça boas práticas desde o início.

Aplicações Corporativas

- Grandes empresas utilizam essa separação arquitetural.
- Sistemas financeiros e de e-commerce dependem dessa organização.
- Ela garante estabilidade em sistemas críticos.

Comunicação Interna Controlada

- Cada camada expõe apenas o necessário.
- Não se deve acessar o banco diretamente pelo Controller.
- Essa disciplina preserva integridade arquitetural.

Manutenção a Longo Prazo

- Sistemas corporativos vivem por muitos anos.
- Arquitetura em camadas facilita manutenção evolutiva.
- Ela reduz retrabalho em grandes refatorações.

Organização de Pacotes

- Projetos costumam organizar pacotes por camada.
- Isso melhora legibilidade do código.
- Também facilita navegação e compreensão estrutural.

Documentação Implícita

- Uma boa divisão em camadas já comunica intenção arquitetural.
- Mesmo sem documentação formal, a estrutura orienta o desenvolvedor.
- Isso melhora o onboarding de novos membros.

Separação de Preocupações

- Separar preocupações significa dividir diferentes tipos de responsabilidade.
- Requisições HTTP não devem se misturar com lógica de persistência.
- Essa divisão torna o sistema mais previsível.

Fluxo Conceitual Completo

- Cliente envia requisição.
- Controller interpreta.
- Service aplica regra.
- Repository acessa banco.
- Resposta retorna ao cliente.

Esse fluxo mantém ordem e controle.

Camadas como Contratos

- Cada camada funciona como um contrato de responsabilidade.
- O Controller promete lidar com requisições.
- O Service promete aplicar lógica.
- O Repository promete persistir dados.

Impacto na Qualidade do Software

- Arquitetura organizada reduz erros estruturais.
- Facilita revisão de código.
- Contribui para padrões consistentes de desenvolvimento.

Base para Microsserviços

- Mesmo em microsserviços, a organização interna segue camadas.
- Cada serviço mantém sua própria divisão interna.
- Isso preserva clareza mesmo em sistemas distribuídos.

Relação com Boas Práticas

- Arquitetura em camadas está alinhada com princípios SOLID.
- Ela promove responsabilidade única.
- Também favorece inversão de dependências.

Síntese da Estrutura

- Controller → Entrada e saída
- Service → Regras de negócio
- Repository → Persistência
- Model → Representação de dados

Essas quatro camadas estruturam aplicações modernas.

Encerramento

- Você aprendeu como aplicações back-end são organizadas em camadas.
- Compreendeu as responsabilidades conceituais de cada uma.
- Entendeu como essa organização garante modularidade, testabilidade e separação de preocupações.

Essa organização em camadas constitui o alicerce estrutural de aplicações corporativas escaláveis e de fácil manutenção ao longo do tempo.

PUC Minas

© **PUC Minas** • Todos os direitos reservados, de acordo com o art. 184 do Código Penal e com a lei 9.610 de 19 de fevereiro de 1998.
Proibidas a reprodução, a distribuição, a difusão, a execução pública, a locação e quaisquer outras
modalidades de utilização sem a devida autorização da Pontifícia Universidade Católica de Minas Gerais.